

VALIDATION DES DONNÉES D'ENTRÉE

LARAVEL

LES BASES DE LA VALIDATION

Démarrage rapide de la validation

Validation des données d'entrée

Pour découvrir les puissantes fonctionnalités de validation de Laravel, examinons un exemple complet de validation d'un formulaire et d'affichage des messages d'erreur à l'utilisateur. En lisant cet aperçu de haut niveau, vous serez en mesure d'acquérir une bonne compréhension générale de la manière de valider les données des requêtes entrantes à l'aide de Laravel

Définir les routes

Tout d'abord, supposons que nous ayons défini les routes suivantes dans notre fichier **routes/web.php** :

```
use App\Http\Controllers\PostController;  
  
Route::get('/post/create', [PostController::class, 'create']);  
Route::post('/post', [PostController::class, 'store']);
```

La route **GET** affichera un formulaire permettant à l'utilisateur de créer un nouvel article de blog, tandis que la route **POST** stockera le nouvel article de blog dans la base de données.

Création du contrôleur

Ensuite, examinons un contrôleur simple qui gère les requêtes entrantes vers ces routes. Nous allons laisser la méthode **store** vide pour l'instant :

```
class PostController extends Controller
{
    public function create()
    {
        return view('post.create');
    }
    public function store(Request $request)
    { // Validate and store the blog post...
    }
}
```

Écriture de la logique de validation

Nous sommes maintenant prêts à remplir notre méthode **store** avec la logique permettant de valider le nouvel article de blog.

Pour ce faire, nous allons utiliser la méthode **validate** fournie par l'objet **Illuminate\Http\Request**. Si les règles de validation passent, votre code continuera à s'exécuter normalement ; toutefois, si la validation échoue, une exception

Illuminate\Validation\ValidationException sera levée et la réponse d'erreur appropriée sera automatiquement renvoyée à l'utilisateur.

Écriture de la logique de validation

Si la validation échoue lors d'une demande HTTP traditionnelle, une réponse de redirection vers l'URL précédente sera générée.

Pour mieux comprendre la méthode de validation, revenons à la méthode de stockage :

```
public function store(Request $request)
{
    $validated = $request->validate([
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ]);
}
```

Écriture de la logique de validation

Les règles de validation peuvent également être spécifiées sous forme de tableaux de règles au lieu d'une seule chaîne délimitée par des | :

```
$validatedData = $request->validate([  
    'title' => ['required', 'unique:posts', 'max:255'],  
    'body' => ['required'],  
]);
```

En outre, vous pouvez utiliser la méthode **validateWithBag** pour valider une demande et stocker les éventuels messages d'erreur dans un **sac d'erreur nommé** :

```
$validatedData = $request->validateWithBag('post', [  
    'title' => ['required', 'unique:posts', 'max:255'],  
    'body' => ['required'],  
]);
```


Arrêt au premier échec de validation

Parfois, vous pouvez souhaiter arrêter l'exécution des règles de validation sur un attribut après le premier échec de validation. Pour ce faire, affectez la règle **bail** à l'attribut :

```
$request->validate([  
    'title' => 'bail|required|unique:posts|max:255',  
    'body' => 'required',  
]);
```

Dans cet exemple, si la règle **unique** sur l'attribut `title` échoue, la règle **max** ne sera pas vérifiée. Les règles seront validées dans l'ordre où elles sont attribuées.

Attributs imbriqués

Si la demande HTTP entrante contient des données de champs "imbriqués", vous pouvez spécifier ces champs dans vos règles de validation en utilisant la syntaxe **"point"** :

```
$request->validate([  
    'title' => 'required|unique:posts|max:255',  
    'author.name' => 'required',  
    'author.description' => 'required',  
]);
```

Échappement de caractère

En revanche, si le nom de votre champ contient un point littéral, vous pouvez empêcher explicitement qu'il soit interprété comme une syntaxe "**point**" en échappant le point par un **antislash** \:

```
$request->validate([  
    'title' => 'required|unique:posts|max:255',  
    'v1\.0' => 'required',  
]);
```

Affichage des erreurs de validation

Toutes les erreurs de validation et les entrées de la demande seront automatiquement **flashées dans la session**.

Une variable **\$errors** est partagée avec toutes les vues de votre application par le middleware **Illuminate\View\Middleware\ShareErrorsFromSession**.

La variable **\$errors** est toujours définie et peut être utilisée en toute sécurité. La variable **\$errors** sera une instance de **Illuminate\Support\MessageBag**.

Affichage des erreurs de validation

Donc, dans notre exemple, l'utilisateur sera redirigé vers la méthode **create** de notre contrôleur lorsque la validation échoue, ce qui nous permet d'afficher les messages d'erreur dans la vue :

```
<h1>Create Post</h1>

@if ($errors->any())
    <div class="alert alert-danger">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif
```

Personnalisation des messages d'erreur

Les règles de validation intégrées de Laravel ont chacune un message d'erreur qui se trouve dans le fichier **lang/en/validation.php** de votre application. Dans ce fichier, vous trouverez une entrée de traduction pour chaque règle de validation. Vous êtes libre de changer ou de modifier ces messages en fonction des besoins de votre application.

En outre, vous pouvez copier ce fichier dans un autre répertoire de langue de traduction afin de traduire les messages pour la langue de votre application.

Requête XHR et validation

Plusieurs applications reçoivent des requêtes XHR d'un frontend alimenté par JavaScript. Lorsque vous utilisez la méthode de validation pendant une requête XHR, Laravel ne génère pas de réponse de redirection. Au lieu de cela, Laravel génère une réponse **JSON** contenant toutes les erreurs de validation. Cette réponse JSON sera envoyée avec un code d'état HTTP **422**.

La directive @error

Vous pouvez utiliser la directive Blade **@error** pour déterminer rapidement si des messages d'erreur de validation existent pour un attribut donné. Dans une directive **@error**, vous pouvez faire écho à la variable **\$message** pour afficher le message d'erreur :

```
<input id="title" type="text" name="title"
class="@error('title') is-invalid @enderror">

@error('title')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror
```


Repopulation des formulaires

Lorsque Laravel génère une réponse de redirection en raison d'une erreur de validation, le framework flashe automatiquement toutes les données de la requête dans la session. Cela vous permet d'accéder facilement aux données lors de la prochaine requête et de repeupler le formulaire que l'utilisateur a tenté de soumettre.

Pour récupérer l'entrée flashée de la requête précédente, invoquez la méthode **old** sur une instance de **Illuminate\Http\Request**.

La méthode **old** récupérera les données d'entrée flashées précédemment dans la session :

```
$title = $request->old('title');
```

Repopulation des formulaires

Laravel fournit également une helper globale **old**. Si vous affichez une ancienne entrée dans un modèle Blade, il est plus pratique d'utiliser le Helper **old** pour repeupler le formulaire. Si aucune ancienne entrée n'existe pour le champ donné, **null** sera retourné :

```
<input type="text" name="title" value="{{ old('title') }}">
```

Champs facultatifs

Par défaut, Laravel inclut les middlewares **TrimStrings** et **ConvertEmptyStringsToNull** dans la pile globale des middlewares de votre application.

Ces middlewares sont répertoriés dans la pile par la classe **App\Http\Kernel**. Pour cette raison, vous aurez souvent besoin de marquer vos champs de requête "facultatifs" comme **nullable** si vous ne voulez pas que le validateur considère les valeurs nulles comme invalides. Par exemple :

```
<input type="text" name="title" value="{{ old('title') }}">
```

Champs facultatifs

Par exemple :

```
$request->validate([  
    'title' => 'required|unique:posts|max:255',  
    'body' => 'required',  
    'publish_at' => 'nullable|date',  
]);
```

Dans cet exemple, nous spécifions que le champ **publish_at** peut être soit null, soit une représentation valide de la date. Si le modificateur **nullable** n'est pas ajouté à la définition de la règle, le validateur considèrera null comme une date non valide.

REQUÊTES DE FORMULAIRES

Validation des requêtes de formulaires

Création de requêtes de formulaire

Pour des scénarios de validation plus complexes, vous pouvez créer une "demande de formulaire". Les demandes de formulaire sont des classes de demandes personnalisées qui encapsulent leur propre logique de validation et d'autorisation. Pour créer une classe de demande de formulaire, vous pouvez utiliser la commande CLI **Artisan make:request** :

```
php artisan make:request StorePostRequest
```

Création de requêtes de formulaire

Pour des scénarios de validation plus complexes, vous pouvez créer une "requête de formulaire". Les requêtes de formulaire sont des classes de requêtes personnalisées qui encapsulent leur propre logique de validation et d'autorisation. Pour créer une classe de requête de formulaire, vous pouvez utiliser la commande CLI **Artisan** **make:request** :

```
php artisan make:request StorePostRequest
```

La classe de requête de formulaire générée sera placée dans le répertoire **app/Http/Requests**. Chaque requête de formulaire générée par Laravel possède deux méthodes : **authorize** et **rules**.

Création de requêtes de formulaire

La méthode **authorize** est chargée de déterminer si l'utilisateur actuellement authentifié peut effectuer l'action représentée par la requête, tandis que la méthode **rules** renvoie les règles de validation qui doivent s'appliquer aux données de la requête :

```
public function rules()
{
    return [
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ];
}
```


Autoriser les requêtes de formulaires

La classe de demande de formulaire contient également une méthode **authorize**. Dans cette méthode, vous pouvez déterminer si l'utilisateur authentifié a effectivement le droit de mettre à jour une ressource donnée.

```
public function authorize()  
{  
    return $this->user()->can('update', $comment);  
}
```

Personnalisation des messages d'erreur

Vous pouvez personnaliser les messages d'erreur utilisés par la demande de formulaire en remplaçant la méthode **messages**. Cette méthode doit retourner un tableau de paires attribut/règle et leurs messages d'erreur correspondants :

```
public function messages()  
{  
    return [  
        'title.required' => 'A title is required',  
        'body.required' => 'A message is required',  
    ];  
}
```

Préparation des données pour la validation

Si vous devez préparer ou assainir des données de la demande avant d'appliquer vos règles de validation, vous pouvez utiliser la méthode **prepareForValidation** :

```
protected function prepareForValidation()  
{  
    // Préparation des données pour la validation  
}
```

LES VALIDATEURS

Création manuelle de validateurs

Création manuelle de validateurs

Si vous ne voulez pas utiliser la méthode **validate** sur la requête, vous pouvez créer une instance de validateur manuellement en utilisant la façade **Validator**. La méthode **make** de la façade génère une nouvelle instance de validateur :

```
public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ]);
    ...
}
```

Création manuelle de validateurs

Après avoir déterminé si la validation de la demande a échoué, vous pouvez utiliser la méthode **withErrors** pour flasher les messages d'erreur dans la session.

```
if ($validator->fails()) {  
    return redirect('post/create')  
        ->withErrors($validator)  
        ->withInput();  
}
```

```
// Récupérer l'entrée validée...  
$validated = $validator->validated();
```

Redirection automatique

Si vous souhaitez créer manuellement une instance de validateur tout en profitant de la redirection automatique offerte par la méthode **validate** de la requête HTTP, vous pouvez appeler la méthode **validate** sur une instance de validateur existante.

```
Validator::make($request->all(), [  
    'title' => 'required|unique:posts|max:255',  
    'body' => 'required',  
)->validate();
```

Vous pouvez utiliser la méthode **validateWithBag** pour stocker les messages d'erreur dans un sac d'erreur nommé si la validation échoue.

Sacs d'erreurs nommés

Si vous avez plusieurs formulaires sur une même page, vous pouvez souhaiter nommer le **MessageBag** contenant les erreurs de validation, ce qui vous permettra de récupérer les messages d'erreur pour un formulaire spécifique. Pour ce faire, passez un nom comme deuxième argument à **withErrors** :

```
return redirect('register')->withErrors($validator, 'login');
```

Vous pouvez alors accéder à l'instance MessageBag nommée à partir de la variable **\$errors** :

```
{{ $errors->login->first('email') }}
```


Personnalisation des messages d'erreur

Si nécessaire, vous pouvez fournir des messages d'erreur personnalisés qu'une instance de validateur doit utiliser à la place des messages d'erreur par défaut fournis par Laravel. Il existe plusieurs façons de spécifier des messages personnalisés. Premièrement, vous pouvez passer les messages personnalisés comme troisième argument à la méthode **Validator::make** :

```
$validator = Validator::make($input, $rules, $messages = [  
    'required' => 'The :attribute field is required.',  
]);
```

DONNÉES VALIDÉES

Travailler avec des données validées

Travailler avec des données validées

Après avoir validé les données d'une requête entrante à l'aide d'une requête de formulaire ou d'une instance de validateur créée manuellement, vous pouvez souhaiter récupérer les données de la requête entrante qui ont effectivement été validées. Cela peut se faire de plusieurs façons. Premièrement, vous pouvez appeler la méthode **validated** sur une demande de formulaire ou une instance de validateur. Cette méthode renvoie un tableau des données qui ont été validées :

```
$validated = $request->validated();
```

```
$validated = $validator->validated();
```

Travailler avec des données validées

Alternativement, vous pouvez appeler la méthode **safe** sur une demande de formulaire ou une instance de validateur. Cette méthode renvoie une instance de **Illuminate\Support\ValidatedInput**. Cet objet expose les méthodes **only**, **except**, et **all** pour récupérer un sous-ensemble des données validées ou le tableau complet des données validées :

```
$validated = $request->safe()->only(['name', 'email']);
```

```
$validated = $request->safe()->except(['name', 'email']);
```

```
$validated = $request->safe()->all();
```

MESSAGES D'ERREUR

Utilisation des messages d'erreur

Utilisation des messages d'erreur

Après avoir appelé la méthode **errors** sur une instance de validateur, vous recevrez une instance de la classe

Illuminate\Support\MessageBag, qui dispose d'une variété de méthodes pratiques pour travailler avec les messages d'erreur.

La variable **\$errors** qui est automatiquement mise à la disposition de toutes les vues est également une instance de la classe MessageBag.

Récupérer le premier message d'erreur d'un champ

Pour récupérer le premier message d'erreur pour un champ donné, utilisez la méthode **first** :

```
$errors = $validator->errors();  
  
echo $errors->first('email');
```

Récupération de tous les messages d'erreur pour un champ

Si vous devez récupérer un tableau de tous les messages pour un champ donné, utilisez la méthode **get** :

```
foreach ($errors->get('email') as $message) {  
    //  
}
```


Récupérer les messages d'erreur pour tous les champs

Pour récupérer un tableau de tous les messages pour tous les champs, utilisez la méthode **all** :

```
foreach ($errors->all() as $message) {  
    //  
}
```

Déterminer si des messages existent pour un champ

La méthode **has** peut être utilisée pour déterminer si des messages d'erreur existent pour un champ donné :

```
if ($errors->has('email')) {  
    //  
}
```

RÈGLES DE VALIDATION

Règles de validation disponibles

Règles de validation disponibles

Vous trouverez ci-dessous une liste de toutes les règles de validation disponibles et de leur fonction :

<https://laravel.com/docs/12.x/validation#available-validation-rules>

Accepted	Ends With	Not In
Accepted If	Enum	Not Regex
Active URL	Exclude	Nullable
After (Date)	Exclude If	Numeric
After Or Equal (Date)	Exclude Unless	Password
Alpha	Exclude With	Present
Alpha Dash	Exclude Without	Prohibited
Alpha Numeric	Exists (Database)	Prohibited If

LES FICHIERS

Validation des fichiers

Validation des fichiers

Laravel fournit une variété de règles de validation qui peuvent être utilisées pour valider les fichiers téléchargés, comme **mimes**, **image**, **min**, et **max**. Bien que vous soyez libre de spécifier ces règles individuellement lors de la validation des fichiers, Laravel offre également un constructeur de règles de validation de fichiers fluide que vous pouvez trouver pratique :

```
Validator::validate($input, [  
    'attachment' => [ 'required',  
                      File::types(['mp3', 'wav'])->min(1024)->max(12 * 1024),  
    ],  
]);
```

Validation des fichiers

Si votre application accepte les images téléchargées par vos utilisateurs, vous pouvez utiliser la méthode de construction d'image de la règle **File** pour indiquer que le fichier téléchargé doit être une image. En outre, la règle Dimensions peut être utilisée pour limiter les dimensions de l'image :

```
File::image()  
  ->min(1024)  
  ->max(12 * 1024)  
  ->dimensions(Rule::dimensions()->maxWidth(1000)->maxHeight(500)),
```

Types de fichiers

Même si vous devez seulement spécifier les extensions lorsque vous invoquez la méthode `types`, cette méthode valide en fait le type MIME du fichier en lisant le contenu du fichier et en devinant son type MIME. Une liste complète des types MIME et de leurs extensions correspondantes peut être trouvée à l'endroit suivant :

<https://svn.apache.org/repos/asf/httpd/httpd/trunk/docs/conf/mime.types>

MOTS DE PASSE

Validation des mots de passe

Validation des mots de passe

Pour s'assurer que les mots de passe ont un niveau de complexité adéquat, vous pouvez utiliser l'objet **Password** de Laravel :

```
use Illuminate\Support\Facades\Validator;  
use Illuminate\Validation\Rules>Password;  
  
$validator = Validator::make($request->all(), [  
    'password' => ['required', 'confirmed', Password::min(8)],  
]);
```

Validation des mots de passe

L'objet **Password** vous permet de personnaliser facilement les exigences de complexité des mots de passe pour votre application:

```
// Exiger au moins 8 caractères...
```

```
Password::min(8)
```

```
// Exiger au moins une lettre...
```

```
Password::min(8)->letters()
```

```
// Il faut au moins une lettre majuscule et une lettre minuscule....
```

```
Password::min(8)->mixedCase()
```

```
// Exiger au moins un numéro...
```

```
Password::min(8)->numbers()
```

```
// Exiger au moins un symbole...
```

```
Password::min(8)->symbols()
```

Validation des mots de passe

Bien entendu, vous pouvez enchaîner toutes les méthodes dans les exemples ci-dessus :

```
Password::min(8)  
  ->letters()  
  ->mixedCase()  
  ->numbers()  
  ->symbols()
```

VALIDATION PERSONNALISÉES

Règles de validation personnalisées

Utilisation d'objets de règles

L'une des méthodes permettant d'enregistrer des règles de validation personnalisées est l'utilisation d'objets règle. Pour générer un nouvel objet règle, vous pouvez utiliser la commande **Artisan make:rule**. Utilisons cette commande pour générer une règle qui vérifie qu'une chaîne de caractères est en majuscules. Laravel placera la nouvelle règle dans le répertoire **app/Rules**.

```
php artisan make:rule Uppercase --invokable
```

Utilisation d'objets de règles

Une fois la règle créée, nous sommes prêts à définir son comportement. Un objet règle contient une seule méthode : **__invoke**. Cette méthode reçoit le nom de l'attribut, sa valeur et une fonction de rappel qui doit être invoquée en cas d'échec avec le message d'erreur de validation :

```
class Uppercase implements InvokableRule
{
    public function __invoke($attribute, $value, $fail){
        if (strtoupper($value) !== $value) {
            $fail('The :attribute must be uppercase.');
        }
    }
}
```

Utilisation d'objets de règles

Une fois la règle est définie, vous pouvez l'attacher à un validateur en passant une instance de l'objet règle avec vos autres règles de validation :

```
use App\Rules\Uppercase;

$request->validate([
    'name' => ['required', 'string', new Uppercase],
]);
```